REACT NATIVE

React Native 5 — Calculator with Pages

Пошаговое учебное руководство
от условия до проверки и сдачи

Обязательная работа

Учебный комплект • 2026

Что требуется

- Создать отдельные страницы-компоненты Calculator и History.
- Использовать React Navigation Native Stack для переходов.
- Разместить калькулятор и кнопку History на Calculator.
- Вывести историю вычислений через FlatList на History.
- Поддержать сложение и вычитание, некорректный ввод и десятичную запятую.
- Использовать strict TypeScript, тесты и Expo SDK 54.

Что получится

Два типизированных Stack-маршрута с общим `HistoryProvider`: успешные выражения остаются при переходе назад и повторном открытии History в пределах текущего запуска.

Подготовка на macOS

Нужны Node.js 20.19.4+, npm и Expo Go либо simulator. `npm install` выбирает зависимости и обновляет lock-файл при создании; в готовой папке `npm ci` ставит строго уже зафиксированные зависимости.

Создание с нуля

КОД · BASH

```
npx create-expo-app@latest 05-calculator-pages --template expo-template-blank-typescript@sdk-54 --
no-agents-md
cd 05-calculator-pages
npx expo install @react-navigation/native react-native-screens react-native-safe-area-context
react-dom react-native-web
npm install @react-navigation/native-stack@^7
npm install --save-dev jest@29.7.0 jest-expo@~54.0.18 @testing-library/react-native@13.3.3 react-
test-renderer@19.1.0 @types/jest@^29.5.14 @types/react@~19.1.0 typescript@~5.9.2
```

Структура файлов

КОД · TEXT

```
05-calculator-pages/
├── App.tsx
├── index.ts
├── tsconfig.json
├── package.json
├── src/{calculator.ts,navigation-types.ts,history-
context.tsx,CalculatorScreen.tsx,HistoryScreen.tsx}
└── __tests__/{calculator.test.ts,App.test.tsx}
```

`package-lock.json` создаётся npm и не является ручным листингом; так же не печатаются `node_modules` и export.

Пошаговая реализация

package.json

КОД · JSON

```
{
  "name": "fullstack-homework-calculator-pages",
  "version": "1.0.0",
  "main": "index.ts",
  "scripts": {
    "start": "expo start",
    "android": "expo start --android",
    "ios": "expo start --ios",
    "web": "expo start --web",
    "test": "jest --watchAll=false",
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {
    "expo": "~54.0.36",
    "expo-status-bar": "~3.0.9",
    "react": "19.1.0",
    "react-native": "0.81.5",
    "react-dom": "19.1.0",
    "react-native-web": "^0.21.0",
    "@react-navigation/native": "^7.1.22",
    "@react-navigation/native-stack": "^7.8.2",
    "react-native-safe-area-context": "~5.6.2",
    "react-native-screens": "~4.16.0"
  },
  "devDependencies": {
    "@testing-library/react-native": "13.3.3",
    "jest": "29.7.0",
    "jest-expo": "~54.0.18",
    "react-test-renderer": "19.1.0",
    "@types/jest": "^29.5.14",
    "@types/react": "~19.1.0",
    "typescript": "~5.9.2"
  },
  "jest": {
    "preset": "jest-expo"
  },
  "private": true
}
```

`main`, `typecheck`, типы и web-зависимости переводят runtime, compiler и Expo Web на TypeScript/SDK 54.

tsconfig.json

КОД · JSON

```
{
  "extends": "expo/tsconfig.base",
  "compilerOptions": {
    "strict": true,
    "noEmit": true
  }
}
```

Базовый config Expo настраивает JSX; strict запрещает implicit `any`, а `noEmit` оставляет сборку Metro.

src/calculator.ts

КОД · TS

```ts
export type Operation = '+' | '-';

export type ParsedNumber =
  | { ok: true; value: number }
  | { ok: false; error: string };

export type CalculationResult =
  | { ok: true; value: number; expression: string }
  | { ok: false; error: string };

export function parseNumberInput(text: string): ParsedNumber {
  const normalized = text.trim().replace(',', '.');
  const value = Number(normalized);

  return normalized === '' || !Number.isFinite(value)
    ? { ok: false, error: 'Enter a valid number.' }
    : { ok: true, value };
}

export function formatNumber(value: number): string {
  return String(Number(value.toFixed(10)));
}

export function calculate(
  firstText: string,
  secondText: string,
  operation: Operation,
): CalculationResult {
  const first = parseNumberInput(firstText);
  const second = parseNumberInput(secondText);

  if (!first.ok) return first;
  if (!second.ok) return second;

  const value = operation === '+' ? first.value + second.value : first.value - second.value;
  if (!Number.isFinite(value)) {
    return { ok: false, error: 'The result is not a finite number.' };
  }

  return {
    ok: true,
    value,
    expression: `${formatNumber(first.value)} ${operation} ${formatNumber(second.value)} =
${formatNumber(value)}`,
  };
}
```

`Operation` ограничивает кнопки `+/-`; union `ok` безопасно открывает error или success fields. `parseNumberInput` нормализует запятую и отвергает пустоту/NaN/Infinity, `formatNumber` убирает decimal noise, `calculate` охраняет overflow.

src/navigation-types.ts

КОД · TS

```ts
export type RootStackParamList = {
  Calculator: undefined;
  History: undefined;
};
```

Один список routes проверяет имена и props навигации.

src/history-context.tsx

КОД · TSX

```tsx
import { createContext, useContext, useRef, useState, type PropsWithChildren } from 'react';

export type HistoryEntry = {
  id: string;
  expression: string;
};

export type HistoryContextValue = {
  history: HistoryEntry[];
  addHistory: (expression: string) => void;
};

const HistoryContext = createContext<HistoryContextValue | undefined>(undefined);

export function HistoryProvider({ children }: PropsWithChildren): React.JSX.Element {
  const [history, setHistory] = useState<HistoryEntry[]>([]);
  const nextIdRef = useRef(1);

  function addHistory(expression: string): void {
    setHistory((previous) => [
      ...previous,
      { id: String(nextIdRef.current++), expression },
    ]);
  }

  return <HistoryContext.Provider value={{ history, addHistory
}}>{children}</HistoryContext.Provider>;
}

export function useCalculationHistory(): HistoryContextValue {
  const context = useContext(HistoryContext);
  if (context === undefined) {
    throw new Error('HistoryProvider is required.');
  }

  return context;
}
```

`HistoryEntry` даёт FlatList stable key; value описывает API Context. Generic state не даёт `never[]`, ref хранит id между render, hook гарантирует Provider.

src/CalculatorScreen.tsx

КОД · TSX

```tsx
import { useState } from 'react';
import type { NativeStackScreenProps } from '@react-navigation/native-stack';
import { Button, Text, TextInput, View } from 'react-native';
import { calculate, formatNumber, type Operation } from './calculator';
import { useCalculationHistory } from './history-context';
import type { RootStackParamList } from './navigation-types';

type CalculatorScreenProps = NativeStackScreenProps<RootStackParamList, 'Calculator'>;

export default function CalculatorScreen({ navigation }: CalculatorScreenProps): React.JSX.Element
{
  const [firstInput, setFirstInput] = useState('');
  const [secondInput, setSecondInput] = useState('');
  const [errorMessage, setErrorMessage] = useState('');
```

```
  const [resultMessage, setResultMessage] = useState('');
  const { addHistory } = useCalculationHistory();

  function handleCalculation(operation: Operation): void {
    const result = calculate(firstInput, secondInput, operation);
    if (!result.ok) {
      setErrorMessage(result.error);
      return;
    }

    addHistory(result.expression);
    setErrorMessage('');
    setResultMessage(`Result: ${formatNumber(result.value)}`);
  }

  return (
    <View style={{ gap: 12, padding: 16 }}>
      <Text>First number</Text>
      <TextInput
        accessibilityLabel="First number"
        keyboardType="decimal-pad"
        placeholder="e.g. 23"
        value={firstInput}
        onChangeText={setFirstInput}
      />
      <Text>Second number</Text>
      <TextInput
        accessibilityLabel="Second number"
        keyboardType="decimal-pad"
        placeholder="e.g. 12"
        value={secondInput}
        onChangeText={setSecondInput}
      />
      <Button title="ADD" onPress={() => handleCalculation('+')} />
      <Button title="SUBTRACT" onPress={() => handleCalculation('-')} />
      <Button title="VIEW HISTORY" onPress={() => navigation.navigate('History')} />
      {errorMessage ? (
        <Text
          accessibilityLabel="Calculation error"
          accessibilityRole="alert"
          accessibilityLiveRegion="assertive"
        >
          {errorMessage}
        </Text>
      ) : null}
      {resultMessage ? (
        <Text accessibilityLabel="Calculation result" accessibilityLiveRegion="polite">
          {resultMessage}
        </Text>
      ) : null}
    </View>
  );
}
```

Props связаны с Calculator route; controlled inputs остаются string. `handleCalculation` сужает result, append-ит успех и публикует assertive error/polite result; navigation проверена `RootStackParamList`.

src/HistoryScreen.tsx

КОД · TSX

```tsx
import type { NativeStackScreenProps } from '@react-navigation/native-stack';
import { Button, FlatList, Text, View } from 'react-native';
import { useCalculationHistory, type HistoryEntry } from './history-context';
import type { RootStackParamList } from './navigation-types';

type HistoryScreenProps = NativeStackScreenProps<RootStackParamList, 'History'>;

export default function HistoryScreen({ navigation }: HistoryScreenProps): React.JSX.Element {
  const { history } = useCalculationHistory();

  function renderHistoryEntry({ item }: { item: HistoryEntry }): React.JSX.Element {
    return <Text>{item.expression}</Text>;
  }

  return (
    <View style={{ gap: 12, padding: 16 }}>
      {history.length === 0 ? <Text>No calculations yet.</Text> : null}
      <FlatList data={history} keyExtractor={(item) => item.id} renderItem={renderHistoryEntry} />
      <Button title="GO BACK" onPress={() => navigation.goBack()} />
    </View>
  );
}
```

`FlatList` получает типизированные entries и id key; empty branch и `goBack` обслуживают два UI path.

App.tsx

КОД · TSX

```tsx
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import CalculatorScreen from './src/CalculatorScreen';
import HistoryScreen from './src/HistoryScreen';
import { HistoryProvider } from './src/history-context';
import type { RootStackParamList } from './src/navigation-types';

const Stack = createNativeStackNavigator<RootStackParamList>();

export default function App(): React.JSX.Element {
  return (
    <HistoryProvider>
      <NavigationContainer>
        <Stack.Navigator initialRouteName="Calculator">
          <Stack.Screen name="Calculator" component={CalculatorScreen} />
          <Stack.Screen name="History" component={HistoryScreen} />
        </Stack.Navigator>
      </NavigationContainer>
    </HistoryProvider>
  );
}
```

Provider стоит над NavigationContainer, поэтому navigation не стирает Context.

index.ts

КОД · TS

```ts
import { registerRootComponent } from 'expo';
import App from './App';

registerRootComponent(App);
```

Expo регистрирует единый entry для native и web.

__tests__/calculator.test.ts

КОД · TS

```ts
import { calculate, formatNumber, parseNumberInput, type Operation } from '../src/calculator';

test.each<[string, number]>([
  ['12', 12],
  ['-3.5', -3.5],
  ['2,75', 2.75],
])('parseNumberInput(%p) accepts a finite number', (text, value) => {
  expect(parseNumberInput(text)).toEqual({ ok: true, value });
});

test.each(['', '  ', 'word', 'Infinity'])('parseNumberInput(%p) rejects invalid text', (text) => {
  expect(parseNumberInput(text)).toEqual({ ok: false, error: 'Enter a valid number.' });
});

test('calculate returns an expression for successful addition and subtraction', () => {
  expect(calculate('2,5', '1.25', '+')).toEqual({
    ok: true,
    value: 3.75,
    expression: '2.5 + 1.25 = 3.75',
  });
  expect(calculate('23', '12', '-')).toEqual({
    ok: true,
    value: 11,
    expression: '23 - 12 = 11',
  });
  expect(formatNumber(3.75)).toBe('3.75');
});

test.each<[string, string, Operation]>([
  ['1e308', '1e308', '+'],
  ['-1e308', '1e308', '-'],
])('calculate rejects a non-finite arithmetic result for %s %s %s', (first, second, operation) =>
{
  expect(calculate(first, second, operation)).toEqual({
    ok: false,
    error: 'The result is not a finite number.',
  });
});
```

Tuple preserves Operation, while examples cover parse, success and overflow branches.

__tests__/App.test.tsx

КОД · TSX

```tsx
import { fireEvent, render } from '@testing-library/react-native';
import App from '../App';

test('calculates, navigates to FlatList history, and retains the item after returning', () => {
  const ui = render(<App />);

  fireEvent.changeText(ui.getByLabelText('First number'), '23');
  fireEvent.changeText(ui.getByLabelText('Second number'), '12');
  fireEvent.press(ui.getByRole('button', { name: 'ADD' }));

  expect(ui.getByLabelText('Calculation result')).toHaveTextContent('Result: 35');
  expect(ui.getByLabelText('Calculation result')).toHaveProp('accessibilityLiveRegion', 'polite');

  fireEvent.press(ui.getByRole('button', { name: 'VIEW HISTORY' }));
  expect(ui.getByText('23 + 12 = 35')).toBeOnTheScreen();

  fireEvent.press(ui.getByRole('button', { name: 'GO BACK' }));
  fireEvent.press(ui.getByRole('button', { name: 'VIEW HISTORY' }));
  expect(ui.getByText('23 + 12 = 35')).toBeOnTheScreen();
});

test('exposes a calculation validation error as an assertive alert', () => {
  const ui = render(<App />);

  fireEvent.press(ui.getByRole('button', { name: 'ADD' }));

  expect(ui.getByRole('alert', { name: 'Calculation error' })).toHaveProp(
    'accessibilityLiveRegion',
    'assertive',
  );
});
```

Real Stack/Provider test protects context retention and accessibility.

Запуск и тесты

КОД · BASH

```bash
npm ci
npm run typecheck
npm test -- --runInBand
npx expo-doctor
npx expo export --platform web --output-dir dist-web
```

Ручная проверка

Проверьте `23 + 12`, `23 - 12`, `2,5 + 1.25`, пустое поле, History, GO BACK и повторное History.

Типичные ошибки

`Number('')` равен нулю; не используйте array index как key, не создавайте Context в screen и не передавайте ошибочное route name.

Вопросы и ответы

Почему Context? Он даёт общий state двум screens. **Почему FlatList?** Это требование и эффективный renderer списка. **Почему string input?** Он допускает промежуточный незавершённый ввод.

Защита

Покажите цепочку `TextInput → calculate → addHistory → Context → FlatList`, union narrowing и generic Stack.

Чек-лист сдачи

- ☐ Два экрана и Native Stack готовы.
- ☐ История через FlatList сохраняется при navigation.
- ☐ TypeScript/tests/Web export выполнены; нет ключей и generated folders.